

阿里前端开发-AI平台

一面（技术基础） · 1-6

1. 谈谈你对事件循环机制的理解，特别是微任务和宏任务的执行顺序。
2. 介绍一下 SSE（Server-Sent Events）和 WebSocket 的区别，为什么 AI 对话场景多用 SSE？
3. 如何使用 CSS 实现一个类似 AI 正在输入的光标闪烁效果？
4. 手写一个并发请求控制函数，限制同时进行的请求数量。
5. 谈谈 Promise.all 和 Promise.allSettled 的区别及适用场景。
6. 什么是 Token？在前端处理 AI 输出时为什么要关注 Token 长度？

二面（深入原理与项目） · 1-8

1. React 的 Fiber 架构解决了什么问题？在 AI 长文本渲染中有什么优势？
2. 在处理 AI 流式输出（Streaming）时，前端如何优雅地处理 Markdown 渲染？
3. 谈谈 Web Worker 在 AI 前端应用中的使用场景。
4. 如何实现一个高性能的 AI 对话历史列表（虚拟滚动）？
5. 什么是 RAG（检索增强生成）？前端在 RAG 流程中可以做哪些优化？
6. 你们项目里是如何处理 Prompt 模板管理的？前端如何辅助用户写出更好的 Prompt？
7. 谈谈前端监控在 AI 平台中的特殊性，你会关注哪些指标？
8. 如果让你设计一个 AI 组件库，你会如何考虑它的通用性和扩展性？

三面（架构与综合能力） 1-5

1. 假设要从零搭建一个企业级 AI Agent 平台，你会如何设计前端的技术架构？
2. AI 响应慢是普遍问题，作为前端，你有哪些手段可以提升用户感知的“快”？
3. 谈谈你对端侧 AI（Local LLM）在前端发展的看法，它会改变现有的开发模式吗？
4. 在过去的一年里，你在 AI 前端领域遇到的最难的技术挑战是什么？你是如何解决的？
5. 阿里非常看重技术对业务的赋能，你觉得 AI 平台前端的价值核心体现在哪里？

一面（技术基础）

1. 谈谈你对事件循环机制的理解，特别是微任务和宏任务的执行顺序。

难度：★

考点：JavaScript 异步执行机制

答案要点：

事件循环是 JavaScript 处理异步操作的核心机制，通过任务队列的循环调度确保单线程不阻塞。

- 执行栈清空后，优先处理所有微任务队列中的任务。
- 宏任务（如 setTimeout, I/O）在下一轮循环开始时执行。
- 微任务（如 Promise.then, MutationObserver）具有更高的优先级，会在当前宏任务结束前全部清空。
- 浏览器会在每一轮宏任务执行结束后，根据需要触发页面渲染。

常见坑：

- 误认为 Promise 构造函数内部是异步的（实际上是同步执行）。
- 混淆 Node.js 早期版本与浏览器的事件循环差异。

追问：

- 如果在微任务里不断递归产生新的微任务，会发生什么？
- await 后面的代码属于宏任务还是微任务？

2. 介绍一下 SSE（Server-Sent Events）和 WebSocket 的区别，为什么 AI 对话场景多用 SSE？

难度：★★

考点：网络协议、AI 流式传输

答案要点：

SSE 是一种基于 HTTP 的单向推送协议，而 WebSocket 是全双工的通信协议。

- 方向性：SSE 是单向（服务端向客户端），WebSocket 是双向。
- 协议层：SSE 基于标准 HTTP，支持自动重连；WebSocket 需要独立的协议握手。
- 资源消耗：SSE 更加轻量，在 AI 这种只需要服务端持续输出文本的场景下性价比更高。
- 兼容性：SSE 原生支持断线重连和自定义事件类型。

常见坑：

- 忽略 SSE 在 HTTP/1.1 下有最大连接数限制（通常为 6 个）。
- 认为 SSE 可以像 WebSocket 一样发送二进制大数据。

追问：

- 如何解决 SSE 在 HTTP/1.1 下的连接数限制问题？
- 前端如何处理 SSE 的异常中断？

3. 如何使用 CSS 实现一个类似 AI 正在输入的光标闪烁效果？

难度：★

考点：CSS 动画、伪元素

答案要点：

通过伪元素模拟光标，并利用关键帧动画控制透明度或可见性。

- 使用 `::after` 伪元素在内容末尾添加一个垂直线条或方块。
- 定义 `@keyframes` 动画，改变 `opacity` 从 0 到 1。
- 设置 `animation-iteration-count` 为 `infinite` 实现持续闪烁。
- 配合 `steps()` 函数可以实现更具机械感的跳动效果。

常见坑：

- 动画导致页面频繁重排（建议使用 `opacity` 而非 `display`）。
- 光标在多行文本末尾的对齐问题。

追问：

- 如果要实现打字机逐字弹出的效果，CSS 能做到吗？

4. 手写一个并发请求控制函数，限制同时进行的请求数量。

难度：★★

考点：异步编程、Promise 运用

答案要点：

并发控制的核心是维护一个执行队列，当某个任务完成后自动触发下一个任务。

- 接收一个任务数组和最大并发数 `limit`。
- 内部维护一个当前运行中的计数器。
- 使用递归或 `while` 循环在任务完成后补充新任务。
- 返回一个 Promise，在所有任务完成后 `resolve`。

常见坑：

- 无法正确处理任务失败的情况，导致后续任务卡死。
- 没考虑任务数组为空的边界情况。

追问：

- 如何给这个并发控制增加超时重试机制？

5. 谈谈 Promise.all 和 Promise.allSettled 的区别及适用场景。

难度：★

考点：ES6 异步 API

答案要点：

Promise.all 具有原子性，而 Promise.allSettled 会记录每一个 Promise 的最终状态。

- Promise.all：只要有一个失败就立即 reject，适用于逻辑强耦合的任务。
- Promise.allSettled：无论成功失败都会等待全部结束，返回状态数组。

- 场景：AI 平台中批量获取模型列表建议用 allSettled，确保部分接口挂了不影响整体展示。

常见坑：

- 在 Promise.all 中没有对单个 Promise 做 catch，导致整个链路崩溃。

追问：

- 如果想实现只要有一个成功就返回，该用哪个 API？

6. 什么是 Token？在前端处理 AI 输出时为什么要关注 Token 长度？

难度：★★

考点：AI 基础概念、前端性能

答案要点：

Token 是模型处理文本的最小单位，通常一个单词或汉字对应一个或多个 Token。

- 成本控制：API 调用通常按 Token 计费。
- 上下文限制：模型有最大上下文长度（Context Window），前端需计算是否超限。
- 性能影响：Token 越多，模型推理时间越长，前端渲染压力越大。
- 截断策略：前端需要根据 Token 长度决定是否触发历史记录清理。

常见坑：

- 简单地用字符长度（length）等同于 Token 长度。

追问：

- 前端如何离线估算 Token 数量？

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？在 AI 长文本渲染中有什么优势？

难度：★★★

考点：React 原理、调度机制

答案要点：

Fiber 引入了可中断的异步渲染机制，解决了 React 15 在处理大量 DOM 更新时造成的页面卡顿。

- 任务拆分：将渲染工作拆分为多个小工单，利用浏览器空闲时间执行。
- 优先级调度：区分动画、交互和数据更新的优先级。
- AI 场景优势：当 AI 持续流式输出长文本时，Fiber 确保了用户依然可以流畅地滚动页面或点击停止按钮。
- 增量渲染：避免了一次性更新巨大 DOM 树导致的逻辑线程阻塞。

常见坑：

- 误以为 Fiber 提高了渲染速度（实际上是提高了响应速度）。

- 不理解协调（Reconciliation）和提交（Commit）阶段的区别。

追问：

- 哪些操作在 Fiber 架构中是不可中断的？

2. 在处理 AI 流式输出（Streaming）时，前端如何优雅地处理 Markdown 渲染？

难度：★★

考点：流式渲染、性能优化

答案要点：

流式渲染需要解决“非完整语法片段”的解析问题，避免渲染器因标签未闭合而报错。

- 增量解析：使用支持流式的 Markdown 解析器（如 markdown-it 的部分插件或自定义逻辑）。
- 缓冲区处理：对代码块、数学公式（KaTeX）等特殊语法进行缓存，直到接收到闭合标识再统一渲染。
- 性能优化：避免每收到一个字符就全量重新渲染 DOM，采用局部更新或虚拟 DOM diff。
- 自动滚动：处理用户手动向上滚动时停止自动跟随时底部的逻辑。

常见坑：

- 代码块高亮库在流式状态下反复初始化导致闪烁。
- 复杂的嵌套列表在流式输出时结构错乱。

追问：

- 如何处理流式输出中的图片或多媒体标签？

3. 谈谈 Web Worker 在 AI 前端应用中的使用场景。

难度：★★

考点：多线程编程、计算密集型任务

答案要点：

Web Worker 将计算密集型任务从主线程剥离，防止 UI 响应失效。

- 场景 1：前端本地分词或 Token 估算。
- 场景 2：大文件的向量化处理或预处理。
- 场景 3：本地运行轻量级模型（如 Transformers.js）。
- 场景 4：对 AI 输出的大量数据进行复杂的正则匹配或清洗。

常见坑：

- 在 Worker 中尝试操作 DOM。
- 频繁的数据传输（PostMessage）带来的序列化开销。

追问：

- 如何优化 Worker 与主线程之间的大数据传输？（Transferable Objects）

4. 如何实现一个高性能的 AI 对话历史列表（虚拟滚动）？

难度：★★

考点：长列表优化、动态高度计算

答案要点：

AI 对话内容高度不固定且含有复杂组件，传统的固定高度虚拟滚动不适用。

- 动态高度计算：先预估高度，渲染后通过 ResizeObserver 监听真实高度并更新偏移量。
- 缓冲区设计：在可视区上下保留一定数量的预渲染节点，减少白屏。
- 滚动锚定：当 AI 在底部输出时，如果用户在查看上方历史，需要锁定滚动条位置。
- 节点复用：确保复杂的 Markdown 组件在滚动时能被有效复用。

常见坑：

- 滚动过快导致的白屏闪烁。
- 动态高度更新时引起的滚动条跳动。

追问：

- 虚拟列表中的 key 应该如何选择以保证性能？

5. 什么是 RAG（检索增强生成）？前端在 RAG 流程中可以做哪些优化？

难度：★★★

考点：AI 业务链路、前端交互设计

答案要点：

RAG 通过检索外部知识库来补充 LLM 的知识，前端主要负责引用展示和交互引导。

- 引用标注（Citation）：在 AI 回复中点击来源标签，前端需实现平滑跳转并高亮文档对应内容。
- 知识库管理：前端实现文档上传、切片预览、向量化进度展示。
- 检索反馈：展示检索到的原始片段，允许用户对检索结果进行点赞/踩，辅助优化 Embedding 算法。
- 预取逻辑：根据用户提问意图，预加载可能相关的文档资源。

常见坑：

- 引用链接失效或指向错误的文档片段。
- 忽略了多模态 RAG（如图片检索）的展示逻辑。

追问：

- 如何在前端实现 PDF 文档的精准高亮跳转？

6. 你们项目里是如何处理 Prompt 模板管理的？前端如何辅助用户写出更好的 Prompt？

难度：★★

考点：Prompt Engineering、产品思维

答案要点：

Prompt 管理需要兼顾灵活性和安全性，前端通过结构化表单降低用户门槛。

- 变量注入：使用双花括号语法（如 {{text}}），前端提供可视化输入框替换变量。
- 版本控制：前端记录 Prompt 的不同版本，支持对比输出效果。
- 引导优化：提供 Prompt 库、自动补全建议、以及基于模板的快捷指令（Slash Commands）。
- 负向约束：在后台静默拼接 System Prompt，防止用户输入违规内容。

常见坑：

- 变量替换时未进行转义，导致 Prompt 注入攻击。
- Prompt 模板过长导致超过模型输入限制。

追问：

- 如何通过前端手段评测两个 Prompt 的优劣？

7. 谈谈前端监控在 AI 平台中的特殊性，你会关注哪些指标？

难度：★★

考点：监控体系、用户体验指标

答案要点：

除了常规的 FP/LCP，AI 平台更关注交互链路的响应质量。

- TTFT（Time to First Token）：首个字符返回的时间，直接影响用户感知的快慢。
- 生成速度（Tokens per Second）：衡量模型输出的流畅度。
- 异常中断率：SSE 连接断开或模型生成报错的比例。
- 用户反馈率：点赞/点踩比例，作为模型效果评价的重要指标。
- 输入长度分布：分析用户习惯，优化 Token 预估逻辑。

常见坑：

- 只监控接口成功率，忽略了 AI 输出内容为空或复读机的情况。

追问：

- 如何监控 AI 回复中的内容幻觉（Hallucination）？

8. 如果让你设计一个 AI 组件库，你会如何考虑它的通用性和扩展性？

难度：★★★

考点：组件化设计、架构能力

答案要点：

AI 组件库需要解耦 UI 表现与模型逻辑，支持多种交互模式。

- 核心抽象：定义标准的 Message 数据结构（Role, Content, Metadata）。
- 渲染插件化：支持自定义 Markdown 渲染器、代码块主题、多模态附件展示。

- 状态解耦：将对话流管理（发送、重试、停止）封装为 Headless Hooks。
- 交互预设：内置打字机、流式加载、引用跳转、快捷建议等 AI 特色交互。

常见坑：

- 组件与特定后端 API 强耦合，无法切换模型供应商。
- 样式过于固定，难以适配不同产品的 UI 风格。

追问：

- 如何处理组件在移动端和 PC 端的交互差异？

三面（架构与综合能力）

1. 假设要从零搭建一个企业级 AI Agent 平台，你会如何设计前端的技术架构？

难度：★★★

考点：系统架构设计、全局视野

答案要点：

架构设计应遵循模块化和可插拔原则，支撑复杂的任务编排和多模型接入。

- 视图层：基于微前端或模块化方案，支持不同 Agent 技能的动态加载。
- 逻辑层：构建统一的 AI SDK，封装 SSE 通信、Token 计算、上下文管理。
- 编排层：设计可视化的工作流编辑器（Workflow Editor），支持拖拽式配置节点。
- 存储层：本地 IndexedDB 存储对话历史，支持离线搜索和快速回显。
- 安全层：实现敏感词过滤、输入长度校验、API Key 安全托管。

常见坑：

- 忽视了多轮对话中上下文过长导致的内存溢出。
- 编排器逻辑过于复杂，导致前端性能瓶颈。

追问：

- 可视化编排中，如何处理循环引用和死循环逻辑？

2. AI 响应慢是普遍问题，作为前端，你有哪些手段可以提升用户感知的“快”？

难度：★★★

考点：用户体验优化、心理学应用

答案要点：

通过交互设计和预处理手段，掩盖后台推理的延迟。

- 立即反馈：点击发送后立即展示用户消息，并显示“思考中”的骨架屏或动画。
- 预测性加载：根据用户输入的前半段话，预先建立连接或触发检索。
- 渐进式渲染：利用 SSE 确保首字快速出现（优化 TTFT）。

- 交互引导：在等待时展示相关的推荐问题或知识点卡片，转移注意力。
- 结果缓存：对高频重复问题进行前端或 CDN 缓存。

常见坑：

- 动画过于复杂反而增加了 CPU 负担，导致首字渲染更慢。

追问：

- 这种“感知优化”在什么情况下会适得其反？

3. 谈谈你对端侧 AI（Local LLM）在前端发展的看法，它会改变现有的开发模式吗？

难度：★★★

考点：技术趋势、前瞻性思考

答案要点：

端侧 AI 结合 WebGPU 和 WebAssembly，将部分推理任务从云端下放到浏览器。

- 隐私优势：敏感数据无需上传云端，直接在本地处理。
- 成本优势：降低服务端的 GPU 算力压力。
- 离线能力：支持无网环境下的基础 AI 功能（如翻译、摘要）。
- 开发模式变化：前端工程师需要掌握模型量化、权重加载、显存管理等新技能。

常见坑：

- 盲目追求端侧运行，忽略了移动端设备性能和电池损耗。

追问：

- 目前在浏览器运行大模型最大的瓶颈是什么？

4. 在过去的一年里，你在 AI 前端领域遇到的最难的技术挑战是什么？你是如何解决的？

难度：★★★

考点：问题解决能力、项目深度

答案要点：

（此题需结合候选人真实经历，以下为示例）

- 挑战示例：处理超长对话导致的 DOM 节点过多和内存泄漏。
- 解决思路：引入了基于虚拟列表的按需加载方案，并对 Markdown 渲染结果进行了 LRU 缓存。
- 深度体现：不仅解决了渲染卡顿，还通过 Profiler 工具定位到了某个第三方代码高亮库的内存溢出问题。
- 结果：首屏加载速度提升 40%，长对话操作流畅度显著提高。

常见坑：

- 描述过于笼统，没有具体的数据支撑或技术细节。

- 解决方案是常规手段，体现不出“挑战性”。

追问：

- 如果重新做一次，你会尝试什么不同的方案？

5. 阿里非常看重技术对业务的赋能，你觉得 AI 平台前端的价值核心体现在哪里？

难度：★★

考点：价值观、业务理解

答案要点：

前端是 AI 能力与用户价值之间的“最后一公里”，决定了 AI 是否好用。

- 降低门槛：通过 UI/UX 设计将复杂的 Prompt 转化为简单的点击和拖拽。
- 效率提升：利用 AI 辅助编程、自动生成代码或文档，直接提升生产力。
- 交互创新：探索对话式 UI（CUI）与图形化 UI（GUI）的最佳结合点。
- 数据闭环：通过前端埋点收集高质量的人机交互数据，反馈给模型迭代。

常见坑：

- 认为前端只是画页面的，不参与核心 AI 逻辑。

追问：

- 你如何衡量一个 AI 功能上线后的成功与否？